

Erroneous behaviour for missing return from assignment

Contents

1. [Revision history](#)
2. [Summary](#)
3. [Motivation](#)
4. [Design questions](#)
5. [Proposal: flowing off an assignment operator is erroneous](#)
6. [References](#)

Revision history

- P2973R0: This (initial) version.

Summary

We propose to change the behaviour of flowing off the end of an overloaded assignment operator from undefined to erroneous, erroneously returning `*this`.

Motivation

Flowing off the end of an assignment operator overload is a common mistake ([example](#), [exam•ple](#), [ex•am•ple](#), [etc.](#)):

```
struct Foo {  
    Foo& operator=(const Foo& rhs) {  
        x = rhs.x;
```

```
y = rhs.y;  
    // error: forgot "return *this;!"  
};
```

In current C++ (unlike in C) it is undefined behaviour to call a function with non-`void` return type that flows off the end, regardless of whether the result of the function call is used ([stmt.return]).

[P2795R3](#) proposes the definition of “erroneous behaviour” in C++, which is well-defined behaviour that is nonetheless an error. The paper lays out [principles](#) by which one can evaluate whether any particular undefined behaviour could be changed to be erroneous. We believe that flowing off an assignment operator meets those criteria:

- Potential for harm: common optimisations can cause the execution of an assignment operator with missing return to “continue executing” unexpected code.
- Detectable: the lack of return should be detectable at translation time. Common compilers can already warn on this.
- Opt-out: the status quo can be reinstated by inserting `std::unreachable()`, e.g. in branches that are known to the user to not be executed.

A word on the cost of the now-defined behaviour: compilers can currently optimise aggressively based on the occurrence of undefined behaviour; all such optimisations would no longer be allowed. Usually, the original program was meaningless anyway, but one could argue that there may exist complex cases where some parts of a function had undefined behaviour but were known not to be used, which would now be pessimised. In such cases, one can explicitly insert `std::unreachable()` to those parts to restore the original behaviour, and we would even consider this an overall improvement, since it makes the unusual control flow more obvious. There remains a standard argument that in a large project one may be including third-party code that cannot be modified and that is not being used, but if that code contains the error under discussion, then with the proposed change it will produce a more expensive program (e.g. one that is larger). We consider this cost acceptable.

Design questions

There is one major design question that should be discussed: precisely which assignment operators do we want to change?

- Only to special members, that is, overloads like `T& T::operator=(const T&)`, `T& T::operator=(T&&)`, but see [class.copy.assign] for the full specification. This is a small, conservative change.
- Any assignment operator that returns a reference-to-self, like `T& T::operator=(int n)`, and regardless of ref-qualification. Such operators are common in “fluent interfaces”, and it is highly likely that `return *this;` was intended.
- Any assignment that returns something that is *convertible* to a reference-to-self. This would include `Base& Derived::operator=(/*...*/)`.
- Any assignment at all, regardless of return type (as long as the return type is not `void`). Unlike in the other alternatives, inserting an implicit `return *this;` would make some such functions ill-formed, where they compile correctly today (and have no undefined behaviour if control never actually flows off the end in the program). This seems hard to justify.

Proposal: flowing off an assignment operator is erroneous

We propose to change the semantics of flowing off the end of an assignment operator overload. The wording is relative to Working Draft [N4958](#). The wording is a placeholder and implements only the most conservative design, only affecting special members.

Modify [stmt.return, 8.7.4] paragraph 4 as follows:

Flowing off the end of a constructor, a destructor, or a non-coroutine function with a cv `void` return type is equivalent to a `return` with no operand. Flowing off the end of a copy or move assignment operator ([class.copy.assign, 11.4.6]) results in erroneous behaviour and is erroneously equivalent to a return with operand `*this;`. Otherwise, flowing off the end of a function that is neither `main` ([basic.start.main, 6.9.3.1]) nor a coroutine ([dcl.fct.def.coroutine, 9.5.4]) results in undefined behavior.

References

- Thomas Köppe, [P2795R3](#): *Erroneous behaviour for uninitialized reads*
- Thomas Köppe, [N4958](#): *Working Draft, Programming Languages — C++*